Usage Guide for Baseline
v1.1.0

Authors: Jeff Autor (Vertiv), Jeff Hilland (HPE), Mike Raineri (Dell), John Leung (Intel)

Date: 2025-11-04

Version: 1.1.0

Supersedes: 1.0

CONTENTS

1 Version History 4
2 Introduction 5
3 Compliance to Open Compute Project Tenets 6
3.1 Openness 6
3.2 Efficiency 6
3.3 Impact 6
3.4 Scale 6
3.5 Sustainability 6
4 Management Tasks 7
5 Use Cases 9
5.1 Redfish model notes 9
5.2 Get accounts 9
5.3 Get sessions 10
5.4 Get FRU info 11
5.5 Set the asset tag 11
5.6 Get the location LED 12
5.7 Set the location LED 12
5.8 Get chassis status 13
5.9 Get power state 13
5.10 Get power usage 14
5.11 Get power limit 14
5.12 Get temperature 15
5.13 Get temperature thresholds 16
5.14 Get fan speeds 17
5.15 Get fan redundancy 18
5.16 Get system log 20
5.17 Clear system log 21
5.18 Get firmware version 21
5.19 Get controller status 21
5.20 Get network info 22
5.21 Reset controller 24
6 References 25
7 License 26
8 About Open Compute Foundation 27

1 Version History

Date	Version	Author	Description
6/15/2021	1.0	John Leung	Initial Baseline usage guide and profile contribution
11/4/2025	1.1	John Leung	Allow either PowerSubsystem or Power resource
			Allow either ThermalSubsystem or Thermal resource
			Require the StaticNameServers and PowerLimit to be writable

2 Introduction

This document describes the manageability tasks that can be performed on a platform which is conformant to the Baseline Hardware Management API v1.1.

The document shows how the Redfish interface is used to accomplish these tasks. This includes examples of the interaction across the Redfish interface.

The document references the OCP Profile as the normative form of the requirements.

3 Compliance to Open Compute Project Tenets

3.1 Openness

The usage guide and profile were developed on a public OCP GitHub repository. An open-source validator exists for validating conformance of an implementation to the profile.

3.2 Efficiency

A common manageability interface reduces the amount of client-side code required to be written, and enables the remote client to be used with various conformant products.

3.3 Impact

Manageability standards enable the interchange of interoperable products. The baseline profile applies to OCP platforms placed on the OCP Marketplace.

3.4 Scale

Redfish is one of the scalable interface for managing platforms out-of-band used on OCP platforms.

3.5 Sustainability

No impact.

4 Management Tasks

The following table lists the baseline management tasks and their requirements.

OCP's out-of-band platform manageability interface is based on Redfish. Redfish is an international recognized standard, specified by Redfish API Specification [1] and the Redfish Data Model Specification [2].

As a Redfish interface, the baseline requirements on the Redfish model elements has been normatively specified in a corresponding Service Baseline v1.0.0 Profile [4]. The syntax of the profile document is specified in the Redfish Interoperability Profile Specification [5]. Issues with the profile can be filed on the OCP Profile repository.

The profile document can be read by the Redfish Interoperability Interop Validator [3] to test the conformance of an implementation. The validator will auto-generate tests, execute the tests, and create a test report. Issues with the validator can be filed on the Redfish discussion board (redfishforum.com).

Use Case	Management Task	Requirement
Account Management	Get accounts	Mandatory
Service Management	Get sessions	Mandatory
Hardware Inventory	Get FRU info	Mandatory
	Get and Set the Asset Tag	Recommended
Hardware Location	Get location LED	Recommended
	Set location LED	Recommended
Status	Get Chassis status	Mandatory
Power	Get power state	If implemented, mandatory
	Get power usage	Recommended
	Get power limit	Recommended
Temperature	Get temperature	If implemented, mandatory
	Get temperature thresholds	If implemented, recommended
Cooling	Get fan speeds	If implemented, mandatory
	Get fan redundancy	If implemented, recommended

Use Case	Management Task	Requirement
Log	Get log entry	Mandatory
	Clear system log	Recommended
Management Controller	Get firmware version	Mandatory
	Get controller status	Mandatory
	Get network info	Mandatory
	Reset controller	Mandatory

5 Use Cases

This section describes how each task is accomplished by interacting with the Redfish Interface.

5.1 Redfish model notes

Some aspects of the Redfish model should be comprehend before the interaction with the Redfish model shown below is understood.

Redfish models a managed node in terms of its physical, logical and manager aspects:

- The physical aspect is modeled via the Chassis resource
- The logical aspect is modeled via the ComputerSystem or Fabric resources
- The manager aspect is modeled via the Manager resource

The relationship between the above resources are specified by the Links property.

- On the Chassis resource, see the Links.ComputerSystems and Links.Storage, Links.Switches, and Links.ManagedBy array properties
- On the ComputerSystem resource, see the Links.Chassis and Links.ManagedBy array properties
- On the Manager resource, see the Links.ManagerForChassis, Links.ManagerForServers, Links.ManagerForSwitches and Links.ManagerForManagers array properties.

5.2 Get accounts

The accounts on the management controller is obtained from the AccountService resource.

```
GET /redfish/v1/AccountService
```

The response message contains the following fragment.

```
{
    "Accounts": { "@odata.id": "/redfish/v1/AccountService/Accounts" },
    "Roles": { "@odata.id": "/redfish/v1/AccountService/Roles" }
}
```

The Roles property specifies the path to the Roles collection resource. The Redfish specification specifies the Admin, Operator and ReadOnly roles be member resource.

The Account resource represents each account on the management controller and the role associated to the account.

```
Get /redfish/v1/AccountService/Accounts/1
```

The response message contains the following fragment.

```
{
    "Name": "User Account",
    "Enabled": true,
    "Password": null,
    "PasswordChangeRequired": false,
    "UserName": "Administrator",
    "RoleId": "Administrator",
    "Locked": false,
    "Links": {
        "Role": { "@odata.id": "/redfish/v1/AccountService/Roles/Administrator" }
    }
}
```

5.3 Get sessions

The sessions on the management controller is obtained from the SessionService resource.

```
GET /redfish/v1/SessionService
```

The response message contains the following fragment.

```
{
    "ServiceEnabled": true,
    "SessionTimeout": 30,
    "Sessions": { "@odata.id": "/redfish/v1/SessionService/Sessions" }
}
```

The Sessions property specifies the path to the Sessions collection resource. The Redfish service creates a Session resource for each session that is established. The following is a fragment of a Session resource.

```
{
    "@odata.id": "/redfish/v1/SessionService/Sessions/1234567890ABCDEF",
    "Id": "1234567890ABCDEF",
    "Name": "User Session",
    "UserName": "Administrator"
}
```

5.4 Get FRU info

The hardware FRU (field replaceable unit) information is obtained from the Chassis resource

```
GET /redfish/v1/Chassis/Ch-1
```

The response message contains the following fragment. The response contains the hardware inventory properties for manufacturer, model, SKU, serial number, and part number. The AssetTag properties is a client writeable property.

```
{
    "Id": "Ch-1",
    "ChassisType": "Node",
    "Manufacturer": "Contoso"
    "Model": "RackScale_Rack",
    "SKU": "..."
    "SerialNumber": "...",
    "PartNumber": "...",
    "AssetTag": null,
    "IndicatorLED": "Off"
}
```

5.5 Set the asset tag

The value of the asset tag is set by patching to AssetTag property in the Chassis resource.

```
PATCH /redfish/v1/Chassis/Ch-1
```

The PATCH request includes the following message.

```
{
    "AssetTag": "989846353530048"
}
```

On successful completion, the response message contains the Chassis resource.

5.6 Get the location LED

The state of the location LED is obtained by retrieving the Chassis resource.

```
GET /redfish/v1/Chassis/Ch-1
```

The response message contains one of the following two fragments.

```
{
    "IndicatorLED": "Lit"
}
```

Or

```
{
    "LocationIndicatorActive": True
}
```

5.7 Set the location LED

The state of the location LED is set by setting the IndicatorLED or the LocationIndicatorActive property in the Chassis resource. The IndicatorLED property has been deprecated in favor of the LocationIndicatorActive property.

```
PATCH /redfish/v1/Chassis/Ch-1
```

The PATCH request includes one of the following two messages, which corresponds to the property returned in the GET request.

```
{
    "IndicatorLED": "Lit"
}
```

Or

```
{
    "LocationIndicatorActive": True
}
```

5.8 Get chassis status

The status and health of the chassis is obtained by retrieving the Chassis resource.

```
GET /redfish/v1/Chassis/Ch-1
```

The response message contains the following fragment. The Status property contains the State and Health properties.

```
{
    "Status": {
        "State": "Enabled",
        "Health": "OK"
    }
}
```

5.9 Get power state

The power state of the chassis is obtained from the Chassis resource.

```
GET /redfish/v1/Chassis/Ch-1
```

The response message contains the following fragment. The response contains the PowerState property.

```
{
    "PowerState": "On"
}
```

5.10 Get power usage

The power usage can obtained via the PowerSubsystem or Power resource, both are subordinate to the Chassis resource. The Power resource has been deprecated in favor of the PowerSubsystem resource.

The power usage is obtained via the PowerSubsystem resource by retrieving the EnvironmentMetrics resource.

```
GET /redfish/v1/Chassis/Ch-1/PowerSubsystem/EnvironmentMetrics
```

The response message contains the following fragment.

```
{
    "PowerWatts": "215"
}
```

The power usage is obtained via the Power resource by retrieving the Power resource.

```
GET /redfish/v1/Chassis/Ch-1/Power
```

The response message contains the following fragment. The PowerControl property contains the PowerConsumedWatts PowerCapacityWatts properties.

```
{
    "PowerControl": {
        "PowerConsumedWatts": "215"
    }
}
```

5.11 Get power limit

The power limit can be obtained via the PowerSubsystem or Power resource, both are subordinate to

the Chassis resource. The Power resource has been deprecated in favor of the PowerSubsystem resource.

The power usage is obtained via the PowerSubsystem resource by retrieving the EnvironmentMetrics resource.

```
GET /redfish/v1/Chassis/Ch-1/PowerSubsystem/EnvironmentMetrics
```

The response message contains the following fragment.

```
{
    "PowerLimitWatts": "220"
}
```

The power limit is obtained via the Power resource by retrieving the Power resource.

```
GET /redfish/v1/Chassis/Ch-1/Power
```

The response message contains the following fragment. The PowerControl property contains the LimitInWatts and LimitException properties.

```
{
    "PowerControl": {
        "PowerLimit": {
            "LimitInWatts": "220"
        }
    }
}
```

5.12 Get temperature

The temperature can be obtained via the ThermalSubsystem or Thermal resource, both are subordinate to the Chassis resource. The Thermal resource has been deprecated in favor of the ThermalSubsystem resource.

The temperature is obtained via the ThermalSubsystem resource by retrieving the ThermalSubsystem resource.

```
GET /redfish/v1/Chassis/Ch-1/ThermalSubsystem
```

The response message contains the following fragment.

```
{
    "ThermalMetric": {
        "TemperatureSummayCelsius": {
            "Exhaust": 21
        }
    }
}
```

The temperature is be obtained via the Thermal resource by retrieving the Thermal resource.

```
GET /redfish/v1/Chassis/Ch-1/Thermal
```

The response message contains the following fragment. Temperatures is an array property. Each element of the array is identified by a Name property. The ReadingCelsius property contains the temperature.

```
{
    "Temperatures": [
        {
            "Name": "Chassis Exhaust Temp",
            "ReadingCelsius": 21
        }
    ]
}
```

5.13 Get temperature thresholds

The temperature thresholds are obtained from the Thermal resource which is subordinate to Chassis resource.

```
GET /redfish/v1/Chassis/Chassis_1/Thermal
```

The response message contains the following fragment. The threshold properties are optional.

```
{
    "Temperatures": [
        {
            "Name": "Chassis Exhaust Temp",
            "UpperThresholdFatal": "45",
            "UpperThresholdCritical": "40",
            "UpperThresholdNonCritical", "35"
        }
    ]
}
```

5.14 Get fan speeds

The fan speeds can be obtained via the ThermalSubsystem or Thermal resource, both are subordinate to the Chassis resource. The Thermal resource has been deprecated in favor of the ThermalSubsystem resource.

The fan speeds are obtained via the ThermalSubsystem resource by retrieving the Fans collection resource.

```
GET /redfish/v1/Chassis/Ch-1/ThermalSubsystem/Fans
```

The response message contains the following fragment.

```
{
    "Members@odata.count": 2,
    "Members": {
        {
            "@odata.id"://redfish/v1/Chassis/Ch-1/ThermalSubsystem/Fans/Bay1
        },
        {
            "@odata.id"://redfish/v1/Chassis/Ch-1/ThermalSubsystem/Fans/CPU1
        }
    }
}
```

The fan speed of a specific fan is obtained by retrieving the Fan resource

```
GET /redfish/v1/Chassis/Ch-1/ThermalSubsystem/Fans/Bay1
```

The response message contains the following fragment.

```
{
    "SpeedPercent": {
        "Reading": 45,
        "SpeedRPM": 2200"
    }
}
```

The fan speeds can be obtained from the Thermal resource by retrieving the Thermal resource.

```
GET /redfish/v1/Chassis/Ch-1/Thermal
```

The response message contains the following fragment. Within the Fans array property, each array member has a Reading and ReadingUnits property.

```
{
    "Fans": [
        {
            "Name": ""
            "Reading": 300
            "ReadingUnits": "RPM"
        }
    ]
}
```

5.15 Get fan redundancy

Fans which are configured in a redundancy set should be available via the resource model. The fan speeds can be obtained via the ThermalSubsystem or Thermal resource, both are subordinate to the Chassis resource. The Thermal resource has been deprecated in favor of the ThermalSubsystem resource.

The fan redundancy can be obtained via the ThermalSubsystem resource by retrieving the Fans collection resource.

```
GET /redfish/v1/Chassis/Ch-1/ThermalSubsystem/Fans
```

The response message contains the following fragment.

```
{
    "FanRedundancy": [
        {
            "RedundancyType: "NPlusM",
            "RedundancyGroup": {
                {
                    "@odata.id"://redfish/v1/Chassis/Ch-1/ThermalSubsystem/Fans/Bay1
                },
                {
                    "@odata.id"://redfish/v1/Chassis/Ch-1/ThermalSubsystem/Fans/Bay2
                }
            }
        }
    ]
}
```

The fans redundancy can be obtained from Thermal resource.

```
GET /redfish/v1/Chassis/Ch-1/Thermal
```

The response message contains the following fragment. The Redundancy array property contains as list of the redundancies. The redundancy contains a RedundancySet property which contains the members of the redundancy set.

```
{
    "Fans": [
        {
            "@odata.id": "/redfish/v1/Chassis/Ch-1/Thermal#/Fans/0",
            "MemberId": "0",
            "Reading": 300
            "ReadingUnits": "RPM"
        },
        {
            "@odata.id": "/redfish/v1/Chassis/Ch-1/Thermal#/Fans/1",
            "MemberId": "1",
            "Reading": 300
            "ReadingUnits": "RPM"
        }
    ],
    "Redundancy": [
        {
            "@odata.id": "/redfish/v1/Chassis/Ch-1/Thermal#/Redundancy/0",
            "MemberId": "0",
            "RedundancySet": \[
                { "@odata.id": "/redfish/v1/Chassis/Ch-1/Thermal#/Fans/0" },
                { "@odata.id": "/redfish/v1/Chassis/Ch-1/Thermal#/Fans/1" }
```

```
            ],
            "Mode": "N+m",
            "Status": {
                "State": "Disabled",
                "Health": "OK"
            },
            "MinNumNeeded": 1,
            "MaxNumSupported": 2
        }
    ]
}
```

5.16 Get system log

The System's log is retrieved is obtained by retrieving the Log resource which represent the system log.

```
GET /redfish/v1/Systems/CS-1/LogService/Log
```

The response message contains the following fragment. The value of the Entries property is the collection resource for the entries in the log.

```
{
    "Name": "System Log",
    "Entries": {
        "@odata.id": "/redfish/v1/Systems/CS-1/LogServices/Log/Entries"
    }
}
```

The client can get each entry of the log.

```
GET /redfish/v1/Systems/CS-1/LogService/Log/Entries/1
```

The following fragment is

```
{
    "EntryType": "SEL",
    "Severity": "Critical",
    "Created": "2012-03-07T14:44:00Z",
```

```
    "Message": "Temperature threshold exceeded",
}
```

5.17 Clear system log

The System's log is retrieved is obtained by retrieving the Log resource which represent the node's log.

```
POST /redfish/v1/Systems/CS-1/LogService/Log/Actions/LogService.ClearLog
```

5.18 Get firmware version

The version of firmware for the management controller is obtained by retrieving the Manager resource which represents the management controller of interest.

```
GET /redfish/v1/Managers/BMC_1
```

The response contains the following fragment. The FirmwareVersion property contains the firmware version of the BMC.

```
{
    "FirmwareVersion": "1.00"
}
```

5.19 Get controller status

The status of the management controller is obtained by retrieving the Manager resource.

```
GET /redfish/v1/Managers/BMC
```

The response message contains the following fragment. The Status property contains the State and Health properties of the manager.

```
{
    "Status": {
        "State": "Enabled",
        "Health": "OK"
    }
}
```

5.20 Get network info

The network information for the management controller is obtained by retrieving the EthernetInterface resource.

```
GET /redfish/v1/Managers/BMC/EthernetInterface
```

The response message contains the following fragment.

```
{
    "Status": {
        "State": "Enabled",
        "Health": "OK"
    }.
    "MacAddress": "1E:C3:DE:6F:1E:24",
    "SpeedMbps": 100,
    "InterfaceEnabled": true,
    "LinkStatus": "LinkUp"
    "HostName": "MyHostName",
    "FQDN": "MyHostName.MyDomainName.com",
    "IPv4Addresses": [
        {
            "Address": "192.168.0.10",
            "SubnetMask": "255.255.252.0",
            "AddressOrigin": "DHCP",
            "Gateway": "192.168.0.1"
        }
    ],
    "NameServers": [
        "192.168.200.10",
        "192.168.150.1",
        "fc00:1234:100::2500"
    ]
}
```

The response message may also contain properties from the following fragment.

```
{
    "StaticNameServers": [
        "192.168.150.1",
        "fc00:1234:200:2500"
    ],
    "DHCPv4": {
        "DHCPEnabled": true,
        "UseDNSServers": true,
        "UseGateway": true,
        "UseNTPServers": false,
        "UseStaticRoutes": true,
        "UseDomainName": true,
        "FallbackAddress": "Static"
    },
    "IPv4StaticAddresses: [
        {
            "Address": "192.168.88.130",
            "SubnetMask": "255.255.0.0",
            "Gateway": "192.168.0.1"
        }
    ],
    "DHCPv6": {
        "OperatingMode": "Stateful",
        "UseDNSServers": true,
        "UseDomainName": false,
        "UseNTPServers": false,
        "UseRapidCommit": false
    },
    "IPv6Addresses": [
        {
            "Address": "2001:1:3:5::100",
            "PrefixLength": 64,
            "AddressOrigin": "DHCPv6",
            "AddressState": "Preferred"
        }
    ],
    "IPv6AddressPolicyTable": [
        {
            "Prefix": "::1/128",
            "Precedence": 50,
            "Label": 0
        }
    ],
    "IPv6StaticAddresses: [
        {
            "Address": "fc00:1234::a:b:c:d",
            "PrefixLength": 64
        }
    ],
    "IPv6StaticDefaultGateways": [
        {
```

```
            "Address": "fe80::fe15:b4ff:fe97:90cd",
            "PrefixLength": 64
        }
    ]
}
```

5.21 Reset controller

The management controller is reset by performing a POST action.

```
POST /redfish/v1/Manager/BMC/Actions/Manager.Reset
```

The POST request includes the following message. The ResetType property contains type of reset to perform.

```
{
    "ResetType": "ForceRestart"
}
```

6 References

[1] "Redfish API Specification"

[2] "Redfish Data Model Specification"

[3] "Redfish Interop Validator"

[4] "Baseline v1.1.0 Profile"

[5] "Redfish Interoperability Profile Specification"

7 License

OCP encourages participants to share their proposals, specifications and designs with the community. This is to promote openness and encourage continuous and open feedback. It is important to remember that by providing feedback for any such documents, whether in written or verbal form, that the contributor or the contributor's organization grants OCP and its members irrevocable right to use this feedback for any purpose without any further obligation.

It is acknowledged that any such documentation and any ancillary materials that are provided to OCP in connection with this document, including without limitation any white papers, articles, photographs, studies, diagrams, contact information (together, "Materials") are made available under the Creative Commons Attribution-ShareAlike 4.0 International License found here: https://creativecommons.org/licenses/by-sa/4.0/, or any later version, and without limiting the foregoing, OCP may make the Materials available under such terms.

As a contributor to this document, all members represent that they have the authority to grant the rights and licenses herein. They further represent and warrant that the Materials do not and will not violate the copyrights or misappropriate the trade secret rights of any third party, including without limitation rights in intellectual property. The contributor(s) also represent that, to the extent the Materials include materials protected by copyright or trade secret rights that are owned or created by any third-party, they have obtained permission for its use consistent with the foregoing. They will provide OCP evidence of such permission upon OCP's request. This document and any "Materials" are published on the respective project's wiki page and are open to the public in accordance with OCP's Bylaws and IP Policy. This can be found at http://www.opencompute.org/participate/legal-documents/. If you have any questions, please contact OCP.

8 About Open Compute Foundation

The Open Compute Project (OCP) brings at-scale innovations and hyperscaler best practices to all, spanning technology domains from the data center to the edge, and the technology stack from silicon, to systems, to site facilities and services. The international OCP Community is made up of organizations and people from hyperscale and tier-2 cloud data center operators, communications providers, colocation providers, diverse enterprises, and technology vendors.

The OCP Foundation fosters collaboration within the Community to tackle market challenges in an array of OCP Projects that create open contributions such as specifications, designs, and more. The OCP Solution Provider Program then recognizes the application of those contributions in compliant products, solutions, and data center facilities and services with designations like OCP Inspired, OCP Accepted and OCP Ready. With the tenets of openness, impact, efficiency, scale and sustainability, the OCP engages and educates thousands of engineers every year through many events and webinars. Across many initiatives the OCP Foundation and Community are meeting the market today and shaping the future.